

RACER: Reinforced Agent Collaboration for Explainable Reasoning on Knowledge Graphs

Yuwei Lou¹[0009-0002-5058-6401], Hao Hu¹[0009-0001-8277-9876], Yuzhou Jiang¹[0009-0006-5881-8843], Zongfei Zhang²[0009-0000-6702-8807], Liang Wang¹[0000-0001-5444-748X], Jincai Liu³[0009-0003-4411-9847], Jidong Ge¹[0000-0003-1773-0942], and Xianping Tao¹[0000-0002-5536-3891]

State Key Laboratory for Novel Software Technology, Nanjing University, China
 {yuweilou, jiangyuzhou}@smail.nju.edu.cn, {myou, wl, gjd, txp}@nju.edu.cn
 Independent Researcher zhangzongfei007@gmail.com
 Chinaunicom Software Nanjing Branch liujc50@chinaunicom.cn

Abstract. Large Language Models (LLMs) often suffer from hallucination and struggle with complex reasoning tasks requiring multi-hop domain knowledge. While integrating Knowledge Graphs (KGs) provides a structured and verifiable information source, current KG-enhanced LLM paradigms usually rely on single-agent path extraction and fixed prompting, lacking adaptability and facing huge search spaces. To address these challenges, we propose RACER, a Reinforced Agent Collaboration framework for Explainable Reasoning on knowledge graphs. RACER employs a semantic-aware action pruning and teacher-guided reinforcement learning mechanism to efficiently extract high-quality reasoning pathways from large-scale KGs. Furthermore, to mitigate single-path generation pitfalls, we introduce a cross-task accumulated shared memory graph paired with an attention-driven multi-path knowledge refinement module. Finally, RACER orchestrates these components through a four-role multi-agent collaboration system (GraphAgent, TemplateAgent, AnswerAgent, and CriticAgent) to dynamically refine prompts and evaluate answers. Extensive experiments on CommonsenseQA and OpenBookQA datasets demonstrate that RACER significantly outperforms state-of-the-art KG-enhanced LLM baselines with an average improvement of 5%, offering robust and highly interpretable reasoning capabilities.

Keywords: Large Language Models · Knowledge Graphs · Reinforcement Learning · Multi-Agent Systems · Commonsense Reasoning.

1 Introduction

As Large Language Models (LLMs) continue to demonstrate increasingly powerful capabilities [1, 19], agents built upon models such as Gemini 3 and Claude 4.5 have begun to see widespread deployment in real-world applications [3, 10, 18, 21]. Despite the impressive performance exhibited by these model-based agents, hallucination problems [4], inherent to their underlying architecture, inevitably emerge and compromise task accuracy. A particularly critical challenge arises

when LLMs encounter scenarios absent from their training data or involving long-range dependencies between related samples, where their effectiveness often degrades significantly [18, 21].

Retrieval-Augmented Generation (RAG) has been regarded as a promising solution for mitigating hallucinations. By leveraging external knowledge bases relevant to the query, RAG enhances the operational efficacy of LLMs [11, 12, 18], substantially reducing hallucination occurrences and thereby improving task accuracy. However, due to variations in data quality, scattered distribution of query-relevant knowledge within external corpora [12], or the necessity of synthesizing multiple external knowledge sources for correct reasoning, RAG often fails to retrieve knowledge that accurately matches the query. These practical challenges significantly limit the effectiveness of RAG in real-world deployments [21]. Although Microsoft’s GraphRAG [5] advances RAG beyond dense vector retrieval by incorporating knowledge graph structures, the core retrieval process itself does not involve the graph structure in a learnable manner, nor does it incorporate trainable retrieval strategies.

Structured Knowledge Graphs (KGs) [9, 21] store atomic facts as (head, relation, tail) triples, inherently supporting multi-hop reasoning, verifiability, and plug-and-play updates [9, 21, 21, 31]. The knowledge encoded in KGs explicitly represents relational logical pathways from one entity to another [9, 21, 31], offering a novel paradigm for LLMs to execute complex tasks. Current research primarily focuses on incorporating fixed KG knowledge during LLM pre-training [22, 25], while another line of work investigates leveraging structural and logical knowledge within KGs to augment internal LLM reasoning processes, as exemplified by K-BERT [14]. However, many state-of-the-art LLMs, such as GPT, Gemini, and Claude series, remain closed-source and inaccessible. Consequently, recent efforts have shifted toward utilizing structured KG knowledge for prompt engineering with LLMs [13, 21]. For instance, CoK [28] validates the faithfulness of LLM-generated reasoning chains by contrasting them with external KG knowledge; KGR [7] progressively extracts and refines knowledge from KGs to optimize LLM reasoning trajectories; and KnowGPT [33] proposes extracting reasoning paths from KGs through machine learning techniques, combined with templates, to enhance LLM performance.

Despite the promising progress in integrating Knowledge Graphs with LLMs, several critical challenges remain: (1) Enormous search space. The vast scale of KGs renders retrieval inherently difficult. Although some approaches [29, 33] employ machine learning techniques to address this, their efficiency remains limited. (2) Fixed template constraints. Most templates designed for LLM execution rely on handcrafted expert rules. Even in KnowGPT, templates are predominantly pre-designed and dynamically selected via multi-armed bandits during task execution, which demonstrates suboptimal performance in practice. (3) Limited task execution efficiency. While enhancing prompt engineering through KGs presents a compelling direction, most existing approaches delegate tasks to a single LLM, completely overlooking the advantages of Multi-Agent Systems (MAS) research.

To address these challenges, we propose **RACER: Reinforced Agent Collaboration for Explainable Reasoning on Knowledge Graphs**, a task-enhancement framework that integrates reinforcement learning, knowledge graphs, and multi-agent collaboration. RACER leverages reinforcement learning to extract critical reasoning paths from knowledge graphs, refining knowledge to establish strong relevance with the query. Furthermore, it employs multi-agent collaborative procedures to align these paths more effectively with task execution, substantially improving operational efficiency. Our key contributions are summarized as follows:

1. We propose a **semantic-aware action pruning** and **teacher-guided reinforcement learning (RL)** framework for extracting efficient reasoning paths over large-scale knowledge graphs conditioned on query inputs.
2. We design a **cross-task accumulated shared memory graph** that records global statistical information for each edge, integrating this knowledge into path selection and refinement processes. Through a **dual attention mechanism**, we refine multiple candidate paths to generate high-quality natural language knowledge snippets, providing interpretable knowledge support for downstream language models.
3. We construct a **four-role multi-agent collaborative reasoning framework** built upon the aforementioned contributions. Through well-defined division of labor and active feedback mechanisms (including a closed-loop review by the CriticAgent), we achieve end-to-end optimization spanning from path search to knowledge generation and final question answering.
4. We systematically validate the effectiveness of our proposed method across multiple benchmark datasets, achieving approximately **5% performance improvement** over state-of-the-art baselines.

2 Related Work

To address the challenges in complex reasoning tasks, specifically hallucination and logical interpretability in LLMs, researchers have proposed various methods. We broadly categorize them into three main areas: Retrieval-Augmented Generation for LLMs, Integration of Knowledge Graphs (KGs) and LLMs, and Multi-Agent Collaboration Frameworks.

2.1 Retrieval-Augmented Generation for LLMs

Recently, Retrieval-Augmented Generation (RAG) models have been extensively explored to enhance LLMs with external knowledge from text corpora or online sources [11, 12, 18]. Combining LLMs with external knowledge retrieval systems can substantially reduce hallucination. However, these approaches face profound challenges in domain-specific or complex applications: (i) Data quality and scattered distribution. Domain knowledge is often scattered across diverse and unstructured sources [12], leading to potential inconsistencies or errors in the retrieved knowledge [21]. (ii) Knowledge hierarchy and complex searching. Textual

documents typically lack explicit relationships and structured organization, limiting the reasoning and inference capabilities of basic RAG models. Furthermore, finding relevant terminology in huge search spaces can be computationally expensive and time-consuming.

2.2 Integration of KGs and LLMs

KG-enhanced LLMs mitigate these unstructured retrieval issues by leveraging the structured relational knowledge in Knowledge Graphs (KGs) to ground the model’s responses in established facts and verifiably logical pathways [9, 21, 31].

Integrating KGs during Training. Earlier studies adopted heuristic methods to inject knowledge from KGs into the LLMs during pre-training or fine-tuning, such as ERNIE [25] and KnowBERT [22], which incorporate entity embeddings and align them with word representations.

KG Prompting for LLMs. Since most state-of-the-art LLMs (e.g., GPT-4 [19], Claude [1]) are confined to a black-box role via APIs, research focus has recently shifted towards KG prompting that enhances fixed LLMs with structured prompts. For instance, CoK [28] introduces a Chain-of-Knowledge prompting to decompose LLM-generated reasoning chains into evidence triples, verifying them natively against external KGs. KGR [7] uses KGs to retroactively verify the initial draft responses generated by LLMs. More recently, KnowGPT [33] leverages multi-armed bandits to extract paths from KGs and applies context-aware modules for prompt construction.

While KG prompting is promising, the reliance on fixed, manually designed prompt templates restricts flexibility across diverse semantic contexts. Existing KG integration heavily relies on single-LLM executions, completely ignoring the structural advantages of Multi-Agent Systems.

2.3 Multi-Agent Collaboration Frameworks

To overcome the performance limits of single-query LLMs on complex knowledge reasoning, recent studies have begun employing multi-agent frameworks [8, 23, 30], where different agents handle distinct reasoning, validation, and generation modules. Multi-agent systems (MAS) divide sophisticated tasks into cooperative sub-roles, mitigating single-agent failure points and improving overall reliability. For instance, AutoGen [30] provides a generalized framework for conversable agents, while frameworks like MetaGPT [8] and ChatDev [23] assign specialized architectural roles to agents for collaborative problem-solving. In contrast to traditional single-agent KG reasoning models, our proposed RACER explicitly incorporates Reinforced Agent Collaboration to orchestrate path searching, knowledge refinement, and answer generation in a synergized pipeline.

3 Methodology

In this section, we introduce the RACER framework, a reinforced multi-agent collaborative system for explainable reasoning on knowledge graphs. As illustrated in Figure 1, RACER consists of three main components: (1) a Semantic-Pruning Knowledge Graph Reinforcement Learning module that extracts critical reasoning paths; (2) a Shared Memory Graph with Dual Attention that records global path statistics and refines candidate paths into high-quality natural language knowledge; and (3) a Multi-Agent Collaborative Reasoning Framework that orchestrates specialized agents (GraphAgent, TemplateAgent, AnswerAgent, and CriticAgent) to seamlessly execute the entire reasoning and answering pipeline.

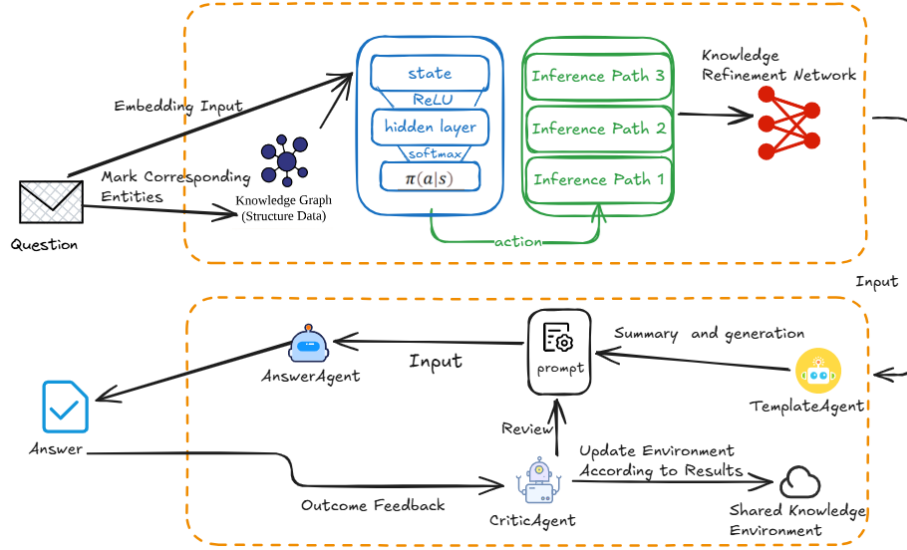


Fig. 1. The overall architecture of the proposed RACER framework. It illustrates the semantic-aware reinforcement learning path searching, the shared memory graph refinement process, and the multi-agent collaboration workflow.

3.1 Semantic-Pruning Knowledge Graph Reinforcement Learning

KnowGPT [33] identifies that an ideal subgraph $\mathcal{G}_{\text{sub}}$ should contain as many relevant source entities $\mathcal{Q}_s$ and target entities $\mathcal{Q}_t$ from the knowledge graph as possible, while maintaining strong relevance to the query context and remaining concise without redundancy, thereby enabling input to LLMs within limited context lengths. However, extracting such subgraphs from large-scale knowledge

graphs is NP-hard, making the identification of $\mathcal{G}_{\text{sub}}$ satisfying all aforementioned criteria computationally intractable.

We posit that paths connecting source entities $\mathcal{Q}_s$ and target entities $\mathcal{Q}_t$ within the ideal subgraph $\mathcal{G}_{\text{sub}}$ consist of multiple logical reasoning chains relevant to the query. Consequently, constructing $\mathcal{G}_{\text{sub}}$ is equivalently formulated as maximizing the identification of such reasoning chains. To efficiently discover these paths, we propose a **semantic-aware action space pruning** and **teacher-guided reinforcement learning** approach. Specifically, we employ Pre-trained Language Models (PLMs) to embed queries, relations, and entities into a unified representation space, thereby enabling efficient semantic matching.

Semantic Action Space Pruning The core idea of semantic pruning is to dynamically compress the action space by computing semantic similarity between candidate actions and the query, thereby eliminating unnecessary search branches. For the candidate action set $A = \{(n_i, r_i)\}_{i=1}^M$ of the current node v_t , we compute the composite score for each action as follows:

$$\text{score}(n_i, r_i) = \alpha \cdot \text{sim}(r_i, \mathbf{q}) + \beta \cdot \text{sim}(n_i, \mathbf{q}) + \gamma \cdot \text{sim}(n_i, \mathbf{g}) \quad (1)$$

where $\mathbf{q}$ and $\mathbf{g}$ denote the dense embedding representations of the query and the target node, respectively, and $\text{sim}(\cdot)$ denotes cosine similarity for measuring semantic relevance. The hyperparameters satisfy $\alpha + \beta + \gamma = 1$, where α, β, γ balance different similarity dimensions.

Based on the scoring function above, we select the top- K actions with the highest scores as the candidate set, significantly reducing the search space. This procedure not only improves search efficiency but also ensures semantic alignment between candidate actions and the query.

Teacher-Guided Policy Gradient Training To further enhance the generalization capability of the policy, we introduce a graph shortest path-based teacher guidance mechanism. Teacher trajectories are computed via the BFS algorithm, providing strong priors for policy training.

For each query, we employ the BFS algorithm to compute the shortest path $P^* = [v_0, v_1, \dots, v_T]$ from the source node v_0 to the target node v_T . The teacher trajectory P^* provides prior information regarding the optimal path.

During training, we follow the next action from the teacher trajectory with high probability p_{teacher} , while sampling from the policy network for the remaining steps. The specific formulation is as follows:

$$\pi(a \mid s_t) = \begin{cases} \mathcal{T}(a \mid s_t), & \text{with probability } p_{\text{teacher}} \\ \pi_{\text{RL}}(a \mid s_t), & \text{with probability } 1 - p_{\text{teacher}} \end{cases} \quad (2)$$

where $\mathcal{T}(a \mid s_t)$ denotes the action probability provided by the teacher trajectory, and $\pi_{\text{RL}}(a \mid s_t)$ represents the reinforcement learning-based policy. The state s_t is mathematically formulated as $s_t = [\mathbf{v}_t; \mathbf{q}; \mathbf{h}_t]$, integrating the current node embedding, the query embedding, and the historical path encoding.

Teacher Supervision Signal. For steps guided by the teacher, we employ a fixed positive reward $R_{\text{teacher}} = 1.0$ as the supervision signal to accelerate policy convergence. This mechanism not only provides strong priors but also significantly enhances the generalization capability of the policy.

Reward Function Design To enable the policy to monotonically approach the target entity in semantic space while providing sufficient exploration incentives, we design a composite reward function comprising multiple components, each serving a specific purpose. The reward formulation aims to balance exploration efficiency and semantic relevance, ensuring effective target node identification.

To penalize unnecessary exploration steps, we introduce a base step penalty r_{step} at each timestep. This penalty term is a negative constant designed to reduce path length and prevent the policy from engaging in protracted exploration:

Relation Similarity Reward. To encourage the policy to select edges semantically relevant to the query, we introduce a relation similarity reward term $r_{\text{rel}} = \lambda_{\text{rel}} \cdot \text{sim}(r_t, \mathbf{q})$, where λ_{rel} is a scaling coefficient. This reward is computed based on the semantic similarity between the traversed relation r_t and the query embedding $\mathbf{q}$.

Goal-Oriented Dense Reward. To guide the policy to monotonically approach the target node, we design a goal-oriented dense reward term r_{goal} . This reward is computed based on the change in semantic similarity between the current node and the target node:

$$r_{\text{goal}} = \lambda_{\text{goal}} \cdot (\text{sim}(v_{t+1}, \mathbf{g}) - \text{sim}(v_t, \mathbf{g})) \quad (3)$$

where $\lambda_{\text{goal}} > 0$ is a hyperparameter controlling the weight of the goal-oriented dense reward, and $\text{sim}(\cdot)$ denotes the cosine similarity function measuring semantic similarity between the embeddings of current node v_t and the target node $\mathbf{g}$. During the inference phase for multiple-choice QA, since the optimal target is unknown, the target representation $\mathbf{g}$ is instantiated by mapping each candidate answer choice to its corresponding graph entity representation, allowing RACER to independently evaluate paths leading towards each candidate option.

Reach Goal Reward. To incentivize the policy to reach the target node efficiently, we introduce a reach goal reward term r_{reach} . When the policy successfully arrives at the target node v_T , a substantial positive reward is granted:

$$r_{\text{reach}} = \begin{cases} \lambda_{\text{reach}}, & \text{if } v_{t+1} = v_T \\ 0, & \text{otherwise} \end{cases} \quad (4)$$

where $\lambda_{\text{reach}} > 0$ is a hyperparameter governing the magnitude of the reach goal reward.

Total Reward Function. Integrating the aforementioned reward components, the total reward function $\mathcal{R}$ is defined as:

$$\mathcal{R} = r_{\text{step}} + r_{\text{rel}} + r_{\text{goal}} + r_{\text{reach}} \quad (5)$$

This reward design not only provides dense reward signals but also ensures that the policy monotonically approaches the target entity in semantic space, significantly improving both reasoning accuracy and efficiency. By adjusting the hyperparameters λ_{step} , λ_{rel} , λ_{goal} , and λ_{reach} , the relative weights of different reward terms can be flexibly balanced to accommodate diverse task requirements.

3.2 Shared Memory Graph and Attention-Driven Knowledge Refinement

Traditional knowledge graph reasoning methods lack effective memory and utilization of historical path quality during path generation. Moreover, single-path search is prone to local optima, failing to provide high-quality auxiliary knowledge and thereby limiting the performance of downstream Large Language Models (LLMs). To address these limitations, we propose a shared memory graph coupled with attention-driven multi-path knowledge refinement method. This approach persistently maintains edge-level statistics and enables intelligent path selection, continuously enhancing the quality of knowledge paths.

Shared Memory Graph The core idea of the shared memory graph is to record the historical performance of each edge, providing global statistical information for path selection to optimize path quality.

For each edge $e = (h, r, t)$ in the knowledge graph, we maintain the following triplet of statistics:

$$\text{stat}(e) = \{\text{succ}(e), \text{fail}(e), \text{reject}(e)\} \quad (6)$$

where $\text{succ}(e)$ denotes the number of times edge e appears in successful paths, $\text{fail}(e)$ denotes the number of times it appears in failed paths, and $\text{reject}(e)$ denotes the number of times it has been rejected during the pruning process.

Edge Quality Scoring. Based on the aforementioned statistics, we compute a quality score for each edge to guide path selection:

$$\text{score}(e) = \frac{\text{succ}(e) + 1}{\text{freq}(e) + 2} - \gamma \cdot \frac{\text{reject}(e)}{\text{total}(e)} \quad (7)$$

where $\text{freq}(e) = \text{succ}(e) + \text{fail}(e)$ and $\text{total}(e) = \text{freq}(e) + \text{reject}(e)$. The hyperparameter γ balances success against rejection rates.

Action Selection Optimization. During action selection, candidate actions are re-ranked according to edge quality scores, prioritizing edges with superior historical performance. This mechanism not only leverages global statistical information but also mitigates local optima.

Multi-Path Generation To generate diverse high-quality paths, we design a multi-path generation mechanism that combines temperature sampling and greedy strategies to balance path diversity and quality.

Given a query $\mathbf{q}$ and a starting node set $\mathcal{S}$, we generate N candidate paths $\mathcal{P} = \{P_1, P_2, \dots, P_N\}$. The path generation process is formulated as:

$$a_t = \begin{cases} \text{sample}(\pi_\theta(\cdot | s_t), \tau), & \text{if } t < \frac{N}{2}, \\ \arg \max_a \pi_\theta(a | s_t), & \text{otherwise,} \end{cases} \quad (8)$$

where $\pi_\theta(\cdot | s_t)$ denotes the action probability distribution of the policy network at state s_t , and τ is the temperature parameter controlling sampling randomness. Temperature sampling is employed during the first half of path generation to enhance diversity, while the greedy strategy is adopted in the second half to ensure quality.

Attention-Driven Path Refinement To extract critical information from multiple candidate paths, we design a lightweight dual-attention path refinement network that filters the most relevant path subset through cross-path relational modeling and query semantic alignment.

Problem Formulation. Given a query q and k candidate paths $\mathcal{P} = \{P_1, \dots, P_k\}$ generated by RL, we learn a refinement function $\mathcal{R} : (q, \mathcal{P}) \rightarrow \mathcal{P}^*$, where $\mathcal{P}^* \subseteq \mathcal{P}$ and $|\mathcal{P}^*| \ll k$, such that the refined path set preserves the most critical information required for answering the query.

Dual-Attention Architecture. We employ pre-trained language models to encode paths and the query:

$$\mathbf{p}_i = \mathbf{W}_p \cdot \text{PLM}(P_i), \quad \mathbf{q} = \mathbf{W}_q \cdot \text{PLM}(q) \quad (9)$$

where $\mathbf{W}_p, \mathbf{W}_q \in \mathbb{R}^{d \times d_{\text{PLM}}}$ are learnable projection matrices.

Cross-Path Self-Attention. To model complementary and redundant relationships among paths, we introduce L layers of multi-head self-attention:

$$\mathbf{H}^{(l)} = \text{MultiHead-SelfAttn}(\mathbf{H}^{(l-1)}), \quad \mathbf{H}^{(0)} = [\mathbf{p}_1; \dots; \mathbf{p}_k] \quad (10)$$

outputting refined path representations $\mathbf{H}^{(L)} \in \mathbb{R}^{k \times d}$.

Query-Path Cross-Attention. We compute semantic matching scores between the query and each path. The query vector is expanded to match the number of paths via a Kronecker product, and the attention weights are acquired:

$$\boldsymbol{\alpha} = \text{softmax} \left(\frac{(\mathbf{1}_k \otimes \mathbf{q})(\mathbf{H}^{(L)})^\top}{\sqrt{d}} \right) \cdot \mathbf{1}_d \in \mathbb{R}^k \quad (11)$$

Importance Scoring and Path Selection. We introduce a learnable scorer to assess the intrinsic importance of paths:

$$\beta = \sigma(\mathbf{W}_2 \cdot \text{ReLU}(\mathbf{W}_1(\mathbf{H}^{(L)})^\top)) \in \mathbb{R}^k \quad (12)$$

The composite score is computed as $s_i = \lambda\alpha_i + (1 - \lambda)\beta_i$, and the Top- k' paths are selected:

$$\mathcal{P}^* = \{P_i \mid i \in \text{Top-}k'(\{s_1, \dots, s_k\})\} \quad (13)$$

where λ is a learnable parameter and k' is determined via validation. The refined path set $\mathcal{P}^*$ is fed into the template generator to provide compact and high-quality context for the LLM.

3.3 Multi-Agent Collaboration Framework

Agent Roles and Collaboration We integrate the aforementioned modules into a collaborative multi-agent system, constructing a four-role multi-agent framework that achieves end-to-end optimization from path search to knowledge generation and final question answering through well-defined division of labor and coordination mechanisms.

GraphAgent. GraphAgent is equipped with the capability to generate knowledge triplet sequences over knowledge graphs using semantic-aware action pruning and teacher-guided reinforcement learning policy networks. Additionally, it integrates Attention-Driven Knowledge Refinement to distill high-quality knowledge triplets.

TemplateAgent. TemplateAgent transforms the high-quality knowledge triplet sequences inferred by GraphAgent into logical knowledge paths, and merges them with the shared memory graph to generate natural language knowledge segment templates comprehensible to large language models.

AnswerAgent. AnswerAgent receives templates generated by TemplateAgent and produces answers according to task requirements. It automatically records the correctness of generated answers and undergoes inspection by CriticAgent.

CriticAgent. CriticAgent examines the entire reasoning and answering process as well as the final results, outputting a decision $d \in \{\text{ACCEPT}, \text{UNSURE}\}$ and a confidence score $c \in [0, 1]$. Furthermore, if the CriticAgent outputs an UNSURE decision, it triggers an active feedback loop, signaling the GraphAgent to explore alternative semantic branches. The failure trajectories are simultaneously penalized in the Shared Memory Graph to prevent redundant cycles, establishing a highly adaptive self-correction operational loop.

4 Experiment

We evaluate RACER on two widely recognized commonsense reasoning datasets and conduct extensive experiments. Our experiments address the following research questions:

RQ1 (Main results): How much does RACER improve over recent KG-enhanced LLM methods?

RQ2 (Ablation Study): Which components contribute critically to RACER’s performance?

4.1 Experimental Setup

Datasets. We select CommonsenseQA [27] and OpenBookQA [17] as benchmarks. CommonsenseQA contains 12,102 multiple-choice questions requiring commonsense knowledge. OpenBookQA contains 5,957 questions requiring elementary science facts. Both use ConceptNet [24] as the background KG.

Baselines. We carefully select baselines from four categories:

- **LM + Fine-tuning:** BERT-base, BERT-large [2], RoBERTa-large [15]
- **KG-enhanced LM:** MHGRN [6], QA-GNN [32], JointLK [26], GreaseLM [34]
- **LLM + Zero-shot:** GPT-3.5 [20], GPT-4 [19], GPT-5, Qwen, GLM, Gemini 3
- **LLM + KG Prompting:** KnowGPT [33], CoK [28], RoG [16], Mindmap [29]

Implementation Details. To demonstrate generalizability, our framework is evaluated using various backbone LLMs (e.g., GPT-5, Qwen, GLM, Gemini 3). All specialized agents are driven by specific systemic prompts over the backbone LLMs, utilizing a few-shot in-context learning paradigm to ensure stable output parsing. To prevent infinite reasoning loops during the CriticAgent’s active feedback phase, we set a maximum collaboration turn limit ($max_turns = 3$). If consensus is not reached within this limit, the AnswerAgent outputs the candidate with the highest initial confidence. The policy network is optimized using Adam with a learning rate of 5×10^{-4} . Semantic pruning parameters, specifically the relation similarity weight and goal similarity weight, are empirically set to 0.25 and 0.35 respectively based on the validation set. The number of selected top paths k' is set to 24, and the maximum path length is limited to 4.

Agent Prompt Templates. To facilitate the multi-agent collaboration while maintaining a compact context footprint, RACER employs specifically designed prompt templates for each agent role, summarized in Table 1. GraphAgent interacts primarily through the reinforcement learning environment and structural APIs.

Table 1. Systemic prompt templates and configurations for the generative agents in RACER.

Agent Role	System Instruction	Input / Output Format
TemplateAgent	You are a template generator. Turn graph-based triples into a short knowledge section for LLMs. MUST NOT answer the question. Write 3-6 concise sentences without mentioning scores.	IN: Question, Options, Triples. OUT: Background knowledge.
AnswerAgent	You are an expert multiple-choice QA solver. Please answer with one letter only based on the provided background knowledge.	IN: Question, Options, Knowledge. OUT: One Letter (A, B, C, D).
CriticAgent	You are an expert QA critic. Analyze whether the model’s generated answer is likely correct, given the graph reasoning path and natural language knowledge.	IN: Question, Options, Triples, Knowledge, Model Answer. OUT: DECISION CONFIDENCE.

4.2 Main Results (RQ1)

Observations. (1) Regardless of the backbone LLM, RACER consistently outperforms both the corresponding Zero-shot versions and existing KG Prompting methods. For example, RACER (GPT-4) achieves 84.7% on CommonsenseQA, surpassing KnowGPT. (2) When utilizing advanced models like GPT-5 or Gemini 3, RACER achieves state-of-the-art performance, with RACER (GPT-5) reaching 98.0% on OpenBookQA, closely approaching human expert levels. (3) The substantial gains observed across diverse models (Qwen, GLM, Gemini 3, etc.) demonstrate that our multi-agent collaboration framework provides a universally effective strategy for structured knowledge injection.

4.3 Ablation Studies (RQ2)

Observations. Removing semantic pruning drops performance the most (-4.6% / -3.7%), followed by teacher guidance (-3.2% / -2.4%). Removing shared memory, attention refinement, or multi-agent collaboration also empirically degrades performance, validating the indispensability of each module.

5 Conclusion

In this paper, we introduced RACER, a novel task-enhancement framework that synergizes reinforcement learning, multi-agent collaboration, and knowledge graphs to address the hallucinations and inference limitations inherent in Large Language Models (LLMs). Specifically, we designed a semantic-aware action pruning mechanism coupled with teacher-guided reinforcement learning to

Table 2. Performance comparison on CommonsenseQA and OpenBookQA.

Model	CommonsenseQA	OpenBookQA
<i>LM + Fine-tuning</i>		
BERT-base	53.0	52.0
BERT-large	61.2	60.2
RoBERTa-large	73.1	64.8
<i>KG-enhanced LM</i>		
MHGRN	71.1	60.0
QA-GNN	73.4	65.0
JointLK	74.5	67.0
GreaseLM	75.0	68.0
<i>LLM + Zero-shot</i>		
GPT-3.5	73.4	60.5
GPT-4	77.2	84.6
GPT-5	82.0	91.0
Qwen3	75.0	86.0
GLM4.7	74.0	87.2
Gemini 3	80.0	90.8
<i>LLM + KG Prompting</i>		
KnowGPT	81.8	92.4
CoK	79.5	90.0
RoG	80.0	91.0
Mindmap	78.5	89.0
<i>Ours (RACER)</i>		
RACER (GPT-4)	84.7	93.2
RACER (GPT-5)	88.2	98.0
RACER (Qwen3)	85.4	92.6
RACER (GLM4.7)	86.5	94.5
RACER (Gemini 3)	87.0	97.5

effectively extract multi-hop reasoning paths from large-scale structured knowledge graphs. Furthermore, by incorporating a cross-task accumulated shared memory graph, we preserve and reuse path-finding statistics to refine knowledge retrieval dynamically. The construction of a four-role multi-agent collaboration mechanism shifts the paradigm from a single inflexible LLM prompt generation step to a highly adaptive, collaborative execution pipeline.

Our systematic evaluation on widely recognized benchmark datasets, including CommonsenseQA and OpenBookQA, demonstrates that RACER significantly outperforms current state-of-the-art baseline models, achieving an average performance improvement of approximately 5%. Moreover, ablation studies validate the criticality of the multi-agent collaboration and RL-guided path extraction components. By grounding the generative outputs in traceable and veritable pathways derived from KGs, RACER provides highly transparent logic verification.

Future work will primarily focus on integrating dynamic knowledge graph updates during active multi-agent reasoning, exploring more sophisticated role

Table 3. Ablation study on RACER (GPT-4) components.

Variant	CommonsenseQA	OpenBookQA
w/o Semantic Pruning	80.1	89.5
w/o Teacher Guidance	81.5	90.8
w/o Shared Memory	82.2	91.3
w/o Attention Refinement	83.0	91.9
w/o Multi-Agent Collaboration	82.8	91.6
Full RACER (GPT-4)	84.7	93.2

allocations, and transferring the RACER framework to domain-specific knowledge reasoning tasks, such as automated medical diagnosis and legal document analysis.

References

1. Anthropic: The claude 3 model family: Opus, sonnet, haiku (2024)
2. Devlin, J., Chang, M.W., Lee, K., Toutanova, K.: Bert: Pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805 (2018)
3. Dong, J., et al.: Clr-bench: Evaluating large language models in college-level reasoning (2024)
4. Dong, J., et al.: Modality-aware integration with large language models for knowledge-based visual question answering (2024)
5. Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Ward, S., Larson, C.: From local to global: A graph rag approach to query-focused summarization. arXiv preprint arXiv:2404.16130 (2024)
6. Feng, Y., Chen, X., Lin, B.Y., Wang, P., Yan, J., Ren, X.: Scalable multi-hop relational reasoning for knowledge-aware question answering. In: EMNLP (2020)
7. Guan, X., et al.: Mitigating large language model hallucinations via autonomous knowledge graph-based retrofitting. AAAI (2024)
8. Hong, S., et al.: Metagpt: Meta programming for a multi-agent collaborative framework. In: ICLR (2024)
9. Hu, L., et al.: A survey of knowledge enhanced pre-trained language models. IEEE TKDE (2023)
10. Kung, T.H., et al.: Performance of chatgpt on usmle: Potential for ai-assisted medical education. PLoS digital health (2023)
11. Lewis, P., et al.: Retrieval-augmented generation for knowledge-intensive nlp tasks. NeurIPS (2020)
12. Li, H., et al.: A survey on retrieval-augmented text generation. arXiv (2022)
13. Liu, P., et al.: Pre-train, prompt, and predict: A systematic survey of prompting methods in nlp. ACM Computing Surveys (2023)
14. Liu, W., et al.: K-bert: Enabling language representation with knowledge graph. AAAI (2020)
15. Liu, Y., Ott, M., Goyal, N., Du, J., Mandar, J., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., Stoyanov, V.: Roberta: A robustly optimized bert pretraining approach. arXiv preprint arXiv:1907.11692 (2019)

16. Luo, L., Li, Y.F., Haffari, G., Pan, S.: Reasoning on graphs: Faithful and interpretable large language model reasoning. arXiv preprint arXiv:2310.01061 (2023)
17. Mihaylov, T., Clark, P., Khot, T., Sabharwal, A.: Can a suit of armor conduct electricity? a new dataset for open book question answering. In: EMNLP (2018)
18. Minaee, S., et al.: Large language models: A survey. arXiv (2024)
19. OpenAI: Gpt-4 technical report (2023)
20. Ouyang, L., et al.: Training language models to follow instructions with human feedback. NeurIPS (2022)
21. Pan, S., et al.: Unifying large language models and knowledge graphs: A roadmap. IEEE TKDE (2024)
22. Peters, M.E., et al.: Knowledge enhanced contextual word representations. EMNLP (2019)
23. Qian, C., et al.: Communicative agents for software development. arXiv preprint arXiv:2307.07924 (2023)
24. Speer, R., Chin, J., Havasi, C.: Conceptnet 5.5: An open multilingual graph of general knowledge. In: AAAI (2017)
25. Sun, Y., et al.: Ernie: Enhanced representation through knowledge integration. arXiv (2019)
26. Sun, Y., Shi, Q., Qi, L., Zhang, Y.: Jointlk: Joint reasoning with language models and knowledge graphs for commonsense question answering. In: ACL (2022)
27. Talmor, A., Herzig, J., Lourie, N., Berant, J.: Commonsenseqa: A question answering challenge targeting commonsense knowledge. arXiv preprint arXiv:1811.00937 (2018)
28. Wang, X., Jiang, Z., Bach, B., et al.: Chain-of-knowledge: Grounding large language models via dynamic knowledge adapting over heterogeneous sources. In: ICLR (2024)
29. Wen, Y., Wang, Z., Sun, J.: Mindmap: Knowledge graph prompting sparks graph of thoughts in large language models. arXiv preprint arXiv:2308.09729 (2023)
30. Wu, Q., et al.: Autogen: Enabling next-gen llm applications via multi-agent conversation. arXiv preprint arXiv:2308.08155 (2023)
31. Yang, L., et al.: Give us the facts: Enhancing large language models with knowledge graphs. arXiv (2024)
32. Yasunaga, M., Ren, H., Bosselut, A., Liang, P., Leskovec, J.: Qa-gnn: Reasoning with language models and knowledge graphs for question answering. In: NAACL (2021)
33. Zhang, Q., Dong, J., Chen, H., Zha, D., Yu, Z., Huang, X.: Knowgpt: Knowledge graph based prompting for large language models. In: NeurIPS (2024)
34. Zhang, X., Bosselut, A., Yasunaga, M., Ren, H., Liang, P., Leskovec, J.: Greaselm: Graph reasoning enhanced language models. In: ICLR (2022)